

Polymorphism:

Polymorphism word is the combination of "poly," which means many + "morphs," which means forms, which together means many forms.

Polymorphism in C++ refers to the ability of objects to take on multiple forms based on their context or usage. It allows different objects to respond to the same message differently.

Polymorphism in C++ is when the behavior of the same object or function is different in different contexts.

This feature in Object-Oriented Programming enhances code flexibility and reusability.

Types of Polymorphism:

Following are the types of polymorphism:

1.Compile Time polymorphism.

Compile-time polymorphism is achieved through function overloading and operator overloading. It is also known as “static ”or “early binding”.

A. Function Overloading

When there are multiple functions with the same name but different parameters, then the functions are said to be **overloaded**, hence this is known as Function Overloading.

Functions can be overloaded by changing the number of arguments or/and changing the type of arguments.

In simple terms, it is a feature of object-oriented programming providing many functions that have the same name but distinct parameters when numerous tasks are listed under one function name.

Example:

```
#include <iostream>
using namespace std;
class Print
{
public:
    void show(int x) {
        cout << "int : " << x << endl;
    }
    void show(char ch) {
        cout << "Char : " << ch << endl;
    }
};
int main() {

    Print p1;
    p1.show(101);
    p1.show('&');
    return 0;
}
```

Constructor overloading is also an example of Compile Time polymorphism.

```
#include <iostream>
using namespace std;

class Student{
public:
    string name;
    Student(){
        cout << "Non-parameterized" << endl;
    }
    Student(string name) {
        this->name = name;
        cout << "Parameterized" << endl;
    }
};
int main() {
    Student s1;
    return 0;
}
```

2. Runtime Polymorphism

This type of polymorphism is achieved by **Function Overriding and** virtual functions. Late binding and dynamic polymorphism are other names for runtime polymorphism. The function call is resolved at runtime in runtime polymorphism.

Run-time polymorphism is achieved through inheritance and virtual functions. The decision about which function to invoke is made at run-time.

Function Overriding:

When a function has the same name and the same function signature (arguments and their data types) in both – the base class and the derived class(es) with a different function definition, this is known as Function Overriding.

When we will such functions from the derived class the base class function gets overridden and child class function gets executed.

Example:

```
class Parent {
public:
    void getInfo() {
        cout << "parent class\n";
    }
};

class Child : public Parent {
public:
    void getInfo() {
        cout << "Child class\n";
    }
};

int main()
{
    Child c1;
    c1.getInfo(); // child class function gets executed
    return 0;
}
```

Virtual Function in C++

A virtual function (also known as virtual methods) is a member function that is declared using the 'virtual' keyword within a base class and is re-defined (overridden) by a derived class.

These functions are used to achieve Runtime polymorphism.

These functions are dynamic in nature. These functions are called in Runtime.

Virtual functions cannot be static.

It is used to tell the compiler to perform dynamic linkage or late binding on the function.

Example:

```
class Parent
{
public:
    virtual void hello() {
        cout << "Hello from Parent\n";
    }
};

class Child : public Parent {
public:
    void hello() {
        cout << "Hello from Child\n";
    }
};

int main() {
    Child c1;
    c1.hello();
    return 0;
}
```

Pure virtual functions:

Pure virtual functions in C++ are functions that do not have an implementation in the base class and must be overridden in any derived class.

A pure virtual function is declared by assigning 0 to a virtual function in the base class. Classes containing at least one pure virtual function are known as abstract classes, and they cannot be instantiated.

Characteristics of Pure Virtual Functions

No Implementation in Base Class: The base class provides no implementation for a pure virtual function.

Forces Derived Class to Override: Any derived class must override the pure virtual function, or the derived class will also become an abstract class.

The syntax for declaring a pure virtual function in a base class is as follows:

```
class Base {
public:
    virtual void display() = 0; // Pure virtual function
};
```

Example:

```
class Shape {
    virtual void draw() = 0; // pure virtual function
};

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a circle\n";
    }
};

int main() {
    Circle c1;
    c1.draw();
    return 0;
}
```